

Capstone Track 1: Video Surveillance with DeepStream

Objective:

Build an AI-powered real-time video analytics pipeline using NVIDIA DeepStream on Jetson or GPU-enabled VM, with TensorRT optimization and Triton-based inference.

Step-by-Step Instructions

Step 1: Define Use Case

Choose a video surveillance scenario:

- Person detection in restricted zones
- Vehicle counting in traffic lanes
- Face detection for entry access

→ Document the use case and its expected outcome.

Step 2: Collect or Simulate Video Input

Use one of the following:

- Public datasets like [Cityscapes](#) or [MOT Challenge](#)
 - Simulated camera stream using GStreamer
 - Real USB/IP camera (for edge device)
-

Step 3: Select Pretrained Model (from NGC)

Choose one DeepStream-compatible model from NGC:

- YOLOv4 / YOLOv5 / SSD for object detection
- FaceNet / OpenPose for facial/body detection

Download and test inference via:

```
ngc registry model list nvidia/tlt_*
```

Step 4: Optimize Model using TensorRT

Convert your model to TensorRT engine format:

```
tlt-converter -k <key> -d <input_dims> -o <output_layer> -e model.engine
```

→ Reduces inference latency and size.

Step 5: Build DeepStream Config and Pipeline

Use `deepstream-app` with a `.txt` config:

```
[primary-gie]
enable=1
model-engine-file=model.engine
batch-size=1
config-file=config_infer.txt
```

→ Customize pipeline components like:

- GStreamer source
 - Inference plugin (nvinfer)
 - Tracker (nvtracker)
 - OSD & video sink
-

Step 6: Containerize the Pipeline

Create a `Dockerfile` using NVIDIA DeepStream base image:

```
FROM nvcr.io/nvidia/deepstream:6.4-triton-multiarch
COPY configs/ /opt/deepstream/configs/
CMD ["deepstream-app", "-c", "/opt/deepstream/configs/app_config.txt"]
```

Build and run:

```
docker build -t deepstream-ai .
docker run --gpus all deepstream-ai
```

Step 7: Deploy Inference with Triton

If separating inference:

- Export TensorRT or ONNX model
- Launch Triton server:

```
docker run --gpus all -p8000:8000 -p8001:8001 \
-v $(pwd)/models:/models nvcr.io/nvidia/tritonserver:24.03-py3 \
tritonserver --model-repository=/models
```

→ Your DeepStream client can now connect to Triton over gRPC or HTTP.

Step 8: Cloud or Edge Integration

Edge: Run on Jetson Orin/Xavier with `deepstream-app`
Cloud: Deploy container on AWS EC2 (g4dn), Azure NC series, or DGX Cloud using Helm/K8s.

Step 9: Monitoring and Logging

Use:

- DeepStream analytics metadata
 - Logging to file or Kafka/MQTT sink
 - Optional: integrate Prometheus & Grafana
-

Step 10: Final Deliverables

-  **Project Report** (2–3 pages with architecture + configs)
 -  **Deployment Pipeline** (container scripts, config files)
 -  **Demo Video** (screen capture showing detection in real-time)
-